

Analysis of different MOR methods for solving nonlinear Elliptic PDEs

1st Maddalena Ghiotti
s332834

2nd Valerio Taralli
s332689

3rd Gaia Saroglia
s332687

Abstract

This project proposes a comparison between three different methods for solving a non-linear parametrized PDE, using the full-order model (FOM) solution as the reference benchmark. Due to the non-affine nature of the problem, computations often become expensive or lead to poor approximations. The aim of the project is therefore to explore how each method reduces computational costs while preserving accuracy.

Since the goal was to obtain a faster way to solve the PDE, the first idea was to exploit model order reduction through the Proper Orthogonal Decomposition method (POD). However, since this method is more suitable for linear models, given the non-affine nature of our problem, its effectiveness was not sufficient. To address the issues arising with the POD method, we introduced Physics-Informed Neural Networks (PINN). This strategy exploits the use of a neural network trained to satisfy both the PDE and its boundary conditions. In this way, we obtained an alternative to solve the PDE that is mesh-free and data-efficient. Finally, the idea was to combine the two strategies by extracting a reduced basis from a collection of high-fidelity solutions, i.e. snapshots, via POD and employing neural networks to approximate the coefficients of the reduced model.

Numerical studies were presented for a nonlinear elliptic parametrized PDE with homogeneous Dirichlet boundary conditions, considering both parameter-dependent and parameter-independent source terms. To evaluate the accuracy of the different approaches, absolute error and relative error were used, while speed-up was computed to measure the gain in terms of computational time compared to the FOM solution. Our analysis allows to show that the POD method is the most accurate, but only partially effective when considering the computational time. On the other hand, PINN is able to achieve good speed-ups at the cost of lower accuracy. Finally, POD-NN, even though it loses accuracy, manages to obtain really remarkable results of computational time.

I. INTRODUCTION

The numerical approximation of nonlinear parametrized partial differential equations (PDEs) using high fidelity techniques, which means solving the full discretized problem, called full-order model (FOM), typically results in very large systems that become computationally expensive and often unfeasible. For this reason, model order reduction (MOR) techniques have gained considerable attention: they are exploited to decrease computational costs while maintaining a good solution approximation.

In the present paper, we study a nonlinear elliptic parametrized equation with homogeneous Dirichlet boundary conditions, on a two-dimensional square domain: both cases with source term dependent on or independent of the parameter are considered. In these two settings, three approaches are implemented¹:

1. the first is the POD-Galerkin method over a finite element FOM;
2. then we focus on parametric PINN, using the same network structure with either parameters independent and dependent source term;

3. in the end, we use, as another reduce method, the hybrid approach of the POD-NN.

Afterwards, the results in terms of computational costs and accuracy are compared using as a benchmark the FOM results.

The outline of the project is as follows: we first introduce the model definition and the methods used; the numerical studies, results and comparisons are next tackled for the given problems; lastly, the conclusions are discussed.

Before continuing, we assume the reader is proficient enough about functional analysis, to avoid explaining all the Hilbert spaces used in the weak formulation (such as $H_0^1(\Omega)$), and about FEM.

II. PROBLEM DEFINITION AND METHODS

In the following, we talk about the discussed PDE and briefly describe the four approaches studied to solve it.

It's noteworthy that, for the sake of notational brevity, the [explicit] dependency on the parameters μ in any function f is represented as a superscript: $f^\mu \equiv f(\mathbf{x}; \mu)$.

A. PDE definition

The non-linear elliptic PDE studied consist of finding

$$u^\mu(\mathbf{x}) : \Omega \times \mathcal{P} \rightarrow \mathbb{R}$$

¹For the acronyms meaning see later sections.

such that it's the solution, for some fixed parameters

$$\boldsymbol{\mu} \equiv (\mu_0, \mu_1) \in \mathcal{P} = [0.1, 1]^2$$

in the parametric square domain, of the following problem with homogeneous Dirichlet boundary conditions:

$$[\text{EP}]^\mu \begin{cases} -\Delta u^\mu + h^\mu = g^\mu + \frac{\mu_0}{\mu_1} & \text{in } \Omega, \\ u^\mu = 0 & \text{on } \partial\Omega, \end{cases}$$

where

$$h^\mu : V \times \mathcal{P} \rightarrow V, \quad (u; \boldsymbol{\mu}) \mapsto h^\mu(u) \equiv \frac{\mu_0}{\mu_1} e^{\mu_1 u}$$

is the non-linear functional term on a space V later defined,

$$g^\mu(\mathbf{x}) : \Omega \times \mathcal{P} \rightarrow \mathbb{R}$$

is the source term to be defined while

$$\mathbf{x} \equiv (x_0, x_1) \in \Omega = (0, 1)^2$$

is the spatial square domain. For the source term two different cases are considered:

C1. independent of $\boldsymbol{\mu}$ with $g^\mu(\mathbf{x}) \equiv g_1(\mathbf{x})$ where:

$$g_1(\mathbf{x}) \equiv 100 \sin(2\pi x_0) \cos(2\pi x_1) \quad (2)$$

C2. dependent on $\boldsymbol{\mu}$ with $g^\mu(\mathbf{x}) \equiv g_2^{\mu_0}(\mathbf{x})$ where:

$$g_2^{\mu_0}(\mathbf{x}) \equiv 100 \sin(2\pi \mu_0 x_0) \cos(2\mu_0 \pi x_1) \quad (3)$$

Lastly the weak formulation of $[\text{EP}]^\mu$ is

$$[\text{VP}]^\mu \begin{cases} \text{Find } u^\mu \in V \text{ such that } \forall v \in V \\ a(u^\mu, v) + k^\mu(u^\mu, v) = F^\mu(v), \end{cases}$$

where $V \equiv H_0^1(\Omega)$

$$a : V \times V \rightarrow \mathbb{R}, \quad (u, v) \mapsto a(u, v) \equiv \int_{\Omega} \nabla u \cdot \nabla v;$$

$$k^\mu : V \times V \rightarrow \mathbb{R}, \quad (u, v) \mapsto k^\mu(u, v) \equiv \int_{\Omega} h^\mu(u) v;$$

$$F^\mu : V \rightarrow \mathbb{R}, \quad v \mapsto F^\mu(v) \equiv \int_{\Omega} \left(g^\mu + \frac{\mu_0}{\mu_1} \right) v.$$

B. Full order model (FOM)

The FOM solution uses the Galerkin method: consider a subspace $V_\delta \subseteq V$ with dimension $N_\delta \equiv \dim\{V_\delta\}_{\delta>0}$, depending on the refinement parameter $\delta > 0$, then $[\text{VP}]^\mu$ becomes

$$[\text{VP}]_\delta^\mu \begin{cases} \text{Find } u_\delta^\mu(\boldsymbol{\mu}) \in V_\delta \text{ such that } \forall v_\delta \in V_\delta \\ a(u_\delta^\mu, v_\delta) + k^\mu(u_\delta^\mu, v_\delta) = F^\mu(v_\delta), \end{cases}$$

in which now u_δ^μ approximate the true solution u^μ . However, due to the nonlinearity of k^μ it's not possible to obtain a matrix form of $[\text{VP}]_\delta^\mu$; hence, it's necessary to use an iterative approach, chosen to be the Newton method: assume to know

the solution of $u_{\delta,n}^\mu$ of the k -th step, the update rule takes the form

$$u_{\delta,n+1}^\mu = u_{\delta,n}^\mu + \delta u_n^\mu. \quad (6)$$

Given the residual of $[\text{VP}]_\delta^\mu$

$$r^\mu(u_\delta^\mu, v_\delta) \equiv a(u_\delta^\mu, v_\delta) + k^\mu(u_\delta^\mu, v_\delta) - F^\mu(v_\delta) = 0,$$

the direction function δu can be found by having $u_{\delta,n+1}^\mu$ such that $r^\mu(u_{\delta,n+1}^\mu, v) = r^\mu(u_{\delta,n}^\mu + \delta u_n^\mu, v) = 0$, which when linearized gives

$$0 = r^\mu(u_{\delta,n}^\mu + \delta u, v_\delta) \approx r^\mu(u_{\delta,n}^\mu, v_\delta) + J_{r,n}^\mu[\delta u_n^\mu, v_\delta] \delta u_n^\mu,$$

that is

$$J_{r,n}^\mu[\delta u_n^\mu, v_\delta] \delta u_n^\mu = -r^\mu(u_{\delta,n}^\mu, v_\delta) \quad \forall v_\delta \in V_\delta, \quad (7)$$

in which $J_{r,n}^\mu[\delta u_n^\mu, v_\delta]$ is the Jacobian of the residual r^μ evaluated in $u_{\delta,n}^\mu$ along the unknown direction δu_n^μ , namely

$$J_{r,n}^\mu[\delta u_n^\mu, v_\delta] \equiv \lim_{h \rightarrow 0} \frac{r^\mu(u_{\delta,n}^\mu + h \delta u_n^\mu, v_\delta) - r^\mu(u_{\delta,n}^\mu, v_\delta)}{h}, \quad (8)$$

assuming r^μ is Gateaux differentiable. From (8), it's then clear that

$$J_{a,n}[\delta u_n^\mu, v_\delta] = \int_{\Omega} \nabla \delta u_n^\mu \nabla v_\delta,$$

for the linearity of $a(u_\delta^\mu, v_\delta)$ w.r.t. u_δ^μ , whereas

$$J_{F,n}^\mu[\delta u_n^\mu; v_\delta] = 0$$

as F^μ it does not depend on u_δ^μ ; thus, the only Jacobian necessary to calculate explicitly is

$$\begin{aligned} J_{k,n}^\mu[\delta u_n^\mu, v_\delta] &= \lim_{h \rightarrow 0} \frac{1}{h} \left[\int_{\Omega} \frac{\mu_0}{\mu_1} \left(e^{\mu_1 h \delta u} - 1 \right) e^{\mu_1 u_{\delta,n}^\mu} v_\delta \right] \\ &= \lim_{h \rightarrow 0} \frac{1}{h} \left[\int_{\Omega} \frac{\mu_0}{\mu_1} (1 + \mu_1 h \delta u - 1) e^{\mu_1 u_{\delta,n}^\mu} v_\delta \right] \\ &= \int_{\Omega} \mu_0 \delta u e^{\mu_1 u_{\delta,n}^\mu} v_\delta, \end{aligned}$$

where $e^{\mu_1 h \delta u} \approx 1 + \mu_1 h \delta u$ is valid only with $h \ll 1$ as is the case when $h \rightarrow 0$. Therefore, (7) becomes

$$[\text{NP}]_\delta^\mu \begin{cases} \text{Find } \delta u_n^\mu \in V_\delta \text{ such that } \forall v_\delta \in V_\delta \\ a(\delta u_n^\mu, v_\delta) + \mu_0 b(\delta u_n^\mu, v_\delta e^{\mu_1 u_{\delta,n}^\mu}) = \\ -a(u_{\delta,n}^\mu, v_\delta) - \frac{\mu_0}{\mu_1} b(e^{\mu_1 u_{\delta,n}^\mu}, v_\delta) + F^\mu(v_\delta), \end{cases}$$

in which

$$b : V \times V \rightarrow \mathbb{R}, \quad (u, v) \mapsto b(u, v) \equiv \int_{\Omega} u v.$$

Introducing now a set of basis function $\{\varphi_k\}_{k=1}^{N_\delta}$ such that $V_\delta = \text{span}\{\varphi_k\}_{k=1}^{N_\delta}$, leads to the linear system

$$\mathbf{S}_n^\mu \delta \mathbf{u}_n^\mu = \mathbf{y}_n^\mu, \quad (10)$$

where

$$\mathbf{S}_n^\mu \equiv \mathbf{S}^\mu(\mathbf{u}_{\delta,n}^\mu) \in \mathbb{R}^{N_\delta \times N_\delta} \quad \text{and} \quad \mathbf{y}_n^\mu \equiv \mathbf{y}^\mu(\mathbf{u}_{\delta,n}^\mu) \in \mathbb{R}^{N_\delta}$$

have the following definition

$$\begin{aligned} \mathbf{S}_n^\mu &\equiv \mathbf{A} + \mu_0 \mathbf{B}_n^{\mu_1}, \\ \mathbf{y}_n^\mu &\equiv -\mathbf{a}_n - \frac{\mu_0}{\mu_1} (\mathbf{b}_n^{\mu_1} - \mathbf{b}) + \mathbf{g}^\mu. \end{aligned}$$

Some remarks ensue from (10):

◇ The update rule in (6) becomes

$$\mathbf{u}_{\delta,n+1}^\mu = \mathbf{u}_{\delta,n}^\mu + \delta \mathbf{u}_n^\mu \quad (11)$$

where now every term is are vectors carrying the weights associated to the basis functions;

- ◇ $\mathbf{A}$ is the stiffness matrix ($A_{ij} \equiv a(\varphi_j, \varphi_i)$), $\mathbf{b}$ is $\mathbf{b}_j \equiv b(1, \varphi_j)$ while $\mathbf{b}_n^{\mu_1}$ is $(\mathbf{b}_n^{\mu_1})_j \equiv b(e^{\mu_1 u_{\delta,n}^\mu}, \varphi_j)$;
- ◇ $\mathbf{A}$, $\mathbf{b}$ and $\mathbf{g}$ are always affine w.r.t. μ ;
- ◇ $\mathbf{B}_n^{\mu_1}$ and $\mathbf{b}_n^{\mu_1}$ are always *not* affine w.r.t. μ_1 ;
- ◇ $\mathbf{g}_n^\mu$ is affine w.r.t. μ_0 if $g \equiv g_1$ but *not* if $g \equiv g_2$.

C. Proper Orthogonal Decomposition (POD)

POD is a strategy that aims to reduce computational costs by solving the system (10) in a reduced space $V_\rho \subseteq V_\delta$ of dimension $N_\rho \ll N_\delta$: assume to know the basis function $\{\xi_k\}_{k=1}^{N_\rho} \in V_\delta$ of such reduced space (i.e. $V_\rho = \text{span}\{\xi_k\}_{k=1}^{N_\rho}$); then it's possible to express each function w.r.t. $\{\varphi_k\}_{k=1}^{N_\delta}$, obtaining the basis matrix

$$\mathbb{B} = [\xi^1 \dots \xi^{N_\rho}] \in \mathbb{R}^{N_\delta \times N_\rho};$$

given a vector $v \in \mathbb{R}^{N_\rho}$, one has $\mathbb{B}v \in \mathbb{R}^{N_\delta}$, that is $\mathbb{B}$ can be seen as a projection from V_ρ to V_δ and the same can be said for the inverse projection with $\mathbb{B}^\top$; thus, defining $\delta \hat{\mathbf{u}}_n^\mu$ such that $\delta \mathbf{u}_n^\mu = \mathbb{B} \delta \hat{\mathbf{u}}_n^\mu$, (10) becomes:

$$\hat{\mathbf{S}}_n^\mu \delta \hat{\mathbf{u}}_n^\mu = \hat{\mathbf{y}}_n^\mu, \quad (12)$$

in which $\hat{\mathbf{S}}_n^\mu \equiv \mathbb{B}^\top \mathbf{S}_n^\mu \mathbb{B}$ and $\hat{\mathbf{y}}_n^\mu \equiv \mathbb{B}^\top \mathbf{y}_n^\mu$. The strategy can be summarized using the following commutative diagram:

$$\begin{array}{ccc} V_\delta & \xrightarrow{\mathbf{S}_n^\mu \delta \mathbf{u}_n^\mu = \mathbf{y}_n^\mu} & V_\delta \\ \mathbb{B}^\top \downarrow & & \uparrow \mathbb{B} \\ V_\rho & \xrightarrow{\hat{\mathbf{S}}_n^\mu \delta \hat{\mathbf{u}}_n^\mu = \hat{\mathbf{y}}_n^\mu} & V_\rho \end{array}$$

where one hopes the reduced path is less computationally demanding than the full one. Of course, as mentioned before, both the iterative and non-affine nature of (10) hinders this chance, though only numerical studies in § can confirm if that's truly the case.

To find $\mathbb{B}$ consider $M \ll N_\delta$, but large enough so that $M \gg N_\rho$, FOM solutions $\mathbf{u}_\delta^n \equiv \mathbf{u}_{\delta,n}^\mu$, $1 \leq n \leq M$, of $[\text{VP}]_\delta^\mu$ with uniformly varying parameters μ , called snapshots, and define

$$\mathbb{U} \equiv [\mathbf{u}_\delta^1 \quad \dots \quad \mathbf{u}_\delta^M]^\top \in \mathbb{R}^{M \times N_\delta}$$

as the matrix whose rows are the snapshots.

Much of the information inside $\mathbb{U}$ is redundant, so one then needs to compress it via the correlation matrix

$$\mathbf{C} \equiv \mathbb{U} \mathbf{A} \mathbb{U}^\top \in \mathbb{R}^{M \times M}; \quad (13)$$

indeed, by solving the eigenvalue problem

$$\mathbf{C} \boldsymbol{\omega}^i = \lambda_i \boldsymbol{\omega}^i, \quad 1 \leq i \leq M, \quad (14)$$

it holds

$$\text{err}_{M,\rho}^2 = \sum_{i=1}^M \|\mathbf{u}_\delta^i - P_\rho(\mathbf{u}_\delta^i)\|_V^2 = \sum_{i=N_\rho+1}^M \lambda_i, \quad (15)$$

where $P_\rho: V_\delta \rightarrow V_\rho$ is the projection operator to the reduced space. In essence (15) says that the most important eigenvectors, i.e. those that retain all significant variance, are those with the largest eigenvalues; and since $\mathbf{C}$ is real, symmetric and positive semi-definite, we can order the all-positive eigenvalues as

$$\lambda_1 \geq \dots \geq \lambda_M \geq 0$$

and take only the first N_ρ so that $\text{err}_{M,\rho}^2 \approx 0$.

However, the eigenvectors $\{\boldsymbol{\omega}^k\}_{k=1}^{N_\rho}$ are still in $\mathbb{R}^M$ while $\{\xi^k\}_{k=1}^{N_\rho}$ lie in $\mathbb{R}^{N_\delta}$; to transform them accordingly it is sufficient to notice how (14) changes with (13):

$$\begin{aligned} (14) &\implies \mathbb{U} \mathbf{A} \mathbb{U}^\top \boldsymbol{\omega}^i = \lambda_i \boldsymbol{\omega}^i, \\ &\implies \mathbf{A} \mathbb{U}^\top \boldsymbol{\omega}^i = \lambda_i \mathbb{U}^\top \boldsymbol{\omega}^i, \\ &\implies \mathbf{A} \boldsymbol{\chi}^i = \lambda_i \boldsymbol{\chi}^i, \quad 1 \leq i \leq M, \end{aligned} \quad (16)$$

where $\boldsymbol{\chi}^i = \mathbb{U}^\top \boldsymbol{\omega}^i$, $1 \leq i \leq N_\rho$, is the transformation sought which also proves $\boldsymbol{\chi}^i$ are the eigenvectors of the stiffness matrix².

The last step comes from linear algebra: in fact, we know thanks to the spectral theorem that both $\boldsymbol{\chi}_{k=1}^{N_\rho}$ and $\{\boldsymbol{\omega}^k\}_{k=1}^{N_\rho}$ are only orthogonal to one another [1, §6.3 p. 286]; hence the reduced basis vectors are obtained by simply normalizing them:

$$\xi^i \equiv \frac{\boldsymbol{\chi}^i}{\|\boldsymbol{\chi}^i\|_V}.$$

²The stiffness matrix shares many properties of $\mathbf{C}$: by definition ($A_{ij} \equiv a(\varphi_j, \varphi_i)$) it's real, symmetric and due to the coercivity of a ($a(v, v) \geq \alpha \|v\|_V^2 \quad \forall v \in V$) it's also definite positive.

D. Physics-Informed Neural Networks (PINN)

In non-linear frameworks, to overcome some POD method's limitations and to exploit the advantages of the division in online and offline phases, the use of the neural networks has opened new opportunities for solving PDEs. In particular, PINN is a flexible approach that directly uses the equations into the training process of a neural network, it incorporates physical laws given by the differential equations into the loss functions, by considering the residual of the PDE, to take advantages in the learning process. This allows PINNs to deal with complex and parametric problems in a data efficient and mesh free way.

The main idea is to incorporate into the loss function both the information obtained from the data and the physical model. The physics-informed term is like a prior knowledge that provides more information to the PINN without the need for more measurements, but only by evaluating the residual of the PDE at additional points in the domain. In our case, we want to approximate the solution of the PDE by considering two contributions to the loss function: one related to boundary information and the other to residual.

The general structure of the parametric PINN method can be summarized as follows:

$$\begin{aligned} \mathcal{F}_{NN}: \quad \Omega \times \mathcal{P} &\rightarrow \mathbb{R}^{N_\delta}, \\ (\mathbf{x}, \boldsymbol{\mu}) &\mapsto \bar{u}^\mu(\mathbf{x}) \approx u^\mu(\mathbf{x}). \end{aligned} \quad (17)$$

where $\mathbf{u}^\mu(\mathbf{x})$ is the vector of coefficients associated with the Galerkin framework, whose entries correspond to the weights of the basis functions. The neural network is a fully-connected feed-forward neural network with n_{HL} hidden layers, each of width n_{H} .

The training process is given by the minimization of the loss function defined as:

$$\text{MSE} = \text{MSE}_b^\mu + \lambda \text{MSE}_p^\mu \quad (18)$$

where MSE is the Mean Squared Error and we compute it for the boundary points using MSE_b^μ , this enforces the condition $u^\mu = 0$, while for the inner points the residual loss MSE_p^μ . Finally, λ is the hyperparameter that balances the trade-off between the two contributions, to be set before the network training.

In particular, by considering our problem, the errors are given by:

$$\begin{aligned} \text{MSE}_b^\mu &= \frac{1}{N_b} \sum_{k=1}^{N_b} \left| \bar{u}^\mu(\mathbf{x}_k^b; \theta) - u^\mu(\mathbf{x}_k^b) \right|^2, \\ \text{MSE}_p^\mu &= \frac{1}{N_p} \sum_{k=1}^{N_p} \left| -\Delta \bar{u}^\mu(\mathbf{x}_k^p) + h^\mu(\bar{u}^\mu(\mathbf{x}_k^p)) - g^\mu(\mathbf{x}_k^p) - \frac{\mu_0}{\mu_1} \right|^2 \end{aligned}$$

for $\mathbf{x}_k^b \in \partial\Omega$ and $\mathbf{x}_k^p \in \Omega$ are the selected points in which we compute the loss and N_b, N_p denote respectively the number of boundary and interior points. This is a mesh-free approach,

so we need to randomly select the boundary points and interior points by fixing the number of them, which is needed to build the network. The hyperparameter λ balances the relative importance of the two contributions. Rather than fixing λ a priori, we dynamically adapt its value during the training. It is updated at each iteration depending on the relative magnitude of MSE_b^μ and MSE_p^μ . In practice, if one loss term dominates the other, λ is modified so as to increase the weight of the weaker contribution, in this way we can balance in a better way the losses and we can be sure that the network is not optimized by relying too much neither over the boundary part nor over the inner part. In this way, the overall stability of the training process is improved.

E. POD-NN hybrid approach

By combining the previous two methods, we obtain a promising paradigm: the POD-NN hybrid approach. This strategy leverages the dimensionality reduction given by POD to obtain a compact representation of the solution space while using a neural network to learn the parametric dependence of the reduced coefficients. The aim of this method is to maintain the accuracy of POD and exploit the benefits of neural networks, providing a computationally efficient alternative to both the MOR and PINN approaches.

As we said in the POD section §II-C, solving the reduced system (12) may still be costly due to the non-affine nature of our problem. The POD-NN strategy avoids the online projection by exploiting the direct projection of snapshots onto the reduced space built with the POD method and then using them to create and train a neural network. Hence, the POD-NN procedure can be naturally divided into two main phases.

We start by exploiting the reduced basis $\{\boldsymbol{\xi}^i\}_{i=1}^{N_\rho}$ that has been constructed in the POD section §II-C. Given a set of M snapshots $\{\mathbf{u}_\delta^{\mu_m}\}_{m=1}^M$ with $\mu_m \in \mathcal{P}$ uniformly varying parameters, the reduced coefficients are computed through orthogonal projection as

$$\hat{\mathbf{u}}^{\mu_m} = (\mathbb{B}^\top \mathbf{A} \mathbb{B})^{-1} \mathbb{B}^\top \mathbf{A} \mathbf{u}_\delta^{\mu_m}, \quad 1 \leq m \leq M,$$

In practice, instead of computing the inverse of the matrix, we solve the linear system

$$(\mathbb{B}^\top \mathbf{A} \mathbb{B}) \hat{\mathbf{u}}^{\mu_m} = \mathbb{B}^\top \mathbf{A} \mathbf{u}_\delta^{\mu_m},$$

which yield the reduced projections of each snapshot. It is important to note that this operation needs to be performed only M times, since the solutions constitute the training set for our neural network:

$$\mathcal{T} = \{(\mu_m, \hat{\mathbf{u}}^{\mu_m}) : 1 \leq m \leq M\}.$$

Then we construct the network: it is a fully-connected feed-forward neural network with input dimension equal to the number of parameters M , output dimension equal to N_ρ , and

n_{HL} hidden layers of width n_{H} . The general structure of the network can be summarized as follows:

$$\mathcal{F}_{\text{PODNN}}: \quad \mathcal{P} \rightarrow \mathbb{R}^{N_{\rho}}, \quad (19)$$

$$\mu^* \mapsto \tilde{\mathbf{u}}^{\mu^*}.$$

where the inputs are the parameters of the training set, while the output is the approximation of the Galerkin projection of the snapshots. To train the network, we need to define a loss function that has to be optimized. The learning process is characterized by the minimization of the Mean Squared Error (MSE) between the predicted coefficients obtained by the network and the reduced coefficients in the training set $\mathcal{T}$:

$$\text{MSE} = \frac{1}{M} \sum_{m=1}^M \|\tilde{\mathbf{u}}^{\mu_m} - \hat{\mathbf{u}}^{\mu_m}\|_2^2. \quad (20)$$

Once trained, we use the network to rapidly predict the reduced coefficients $\hat{\mathbf{u}}^{\mu}$ for any new parameter $\mu \in \mathcal{P}$, avoiding the solution of the reduced system (12).

To sum up, we can compute offline the snapshots collection, apply the POD strategy and train the network. Then, we evaluate the network during the online phase. In this way, we exploit the POD properties and, thanks to the trained neural network, the online phase becomes very fast. In contrast, the offline phase is longer and the accuracy is more difficult to control.

III. SETTINGS AND RESULTS

The entire project was developed in the `Python` programming language, using several key libraries for numerical computation and data processing. In particular, the `NumPy` library was used for basic vector and matrix operations, `GeDiM4Py` was employed to interface with the finite element solver, and `PyTorch` was used for implementing, training, and managing the neural networks in the PINN and POD-NN models.

To simplify the management of repetitive code, a custom library was developed containing a collection of auxiliary functions (e.g., for extracting degrees of freedom, handling inner product matrices, saving models, etc.), with the goal of improving the modularity and readability of the overall workflow.

The two case studies (one with the source term independent of the parameters, and the other with the parameter-dependent source term) were implemented in separate notebooks. Both cases were analyzed using all the proposed methodologies: FOM, POD, PINN, and POD-NN.

A. Models setup

FOM setup

1) POD

The reduction was initially performed by applying the POD method to the snapshots generated during the FOM phase, obtained by solving the PDE for 100 parameters $\mu = (\mu_0, \mu_1)$ sampled uniformly. The discretization of the domain $[0, 1]^2$ was carried out using second-order finite elements, with a maximum element area set to $a_{\text{max}} = 0.0005$, which ensured sufficient resolution for representing the solutions.

The correlation matrix was computed using the inner product induced by the stiffness matrix. The number of modes N was selected by imposing a cumulative energy threshold of $1 - 10^{-7}$, with a maximum of 10 modes considered. Once the reduced orthonormal basis was constructed, Galerkin projections of the snapshots onto the reduced space were computed by solving a linear system for each parameter, avoiding the explicit computation of the inverse of the reduced matrix.

2) PINN

The PINN model is an example of unsupervised learning and was implemented using a fully connected neural network with 5 hidden layers and 50 neurons per hidden layer. The activation function used is `tanh`, chosen for its smoothness and compatibility with partial differential equation problems. The network takes the spatial coordinates (x, y) and the parameters (μ_0, μ_1) as input and outputs an approximation of the scalar solution $u(x, y; \mu_0, \mu_1)$. Before training, the `PyTorch` seed was fixed to 0 to ensure reproducibility.

During training, 10,000 points were randomly generated inside the domain and another 10,000 on the boundary, uniformly distributed. A *mesh-free* approach was adopted, allowing direct sampling of points within and on the boundary of the domain without the need for an explicit mesh. This choice significantly reduces computational costs and enables rapid extraction of a large number of points such as 10,000. Moreover, it allows independent control of the number of interior and boundary points, unlike mesh-based approaches, where increasing boundary resolution implies a simultaneous increase of internal points.

The total loss function is given by the sum of the term associated with the boundary conditions and the PDE residual, weighted by a factor $\lambda = 1$. Optimization was performed using the Adam algorithm, with an initial learning rate of 10^{-3} and a learning rate scheduler based on `ReduceLROnPlateau`, which automatically reduces the learning rate by a factor of 0.75 if the loss stagnates for 5000 consecutive epochs. Training was carried out for a maximum of 500,000 epochs, with early stopping triggered when the loss dropped below a tolerance threshold of 10^{-7} .

After training the PINN model, a saving mechanism was implemented to allow training to be resumed or evaluations to be repeated without re-running the entire process. At the end of training, the following objects are saved to a file:

◇ the neural network weights (`modelStateDict`),

- ◇ the optimizer state (optimiserStateDict),
- ◇ the learning rate scheduler state (schedulerStateDict),
- ◇ the final loss value,
- ◇ the structure of the network (number of layers and neurons per layer),
- ◇ and the last logged training step.

These objects are serialized into a .pth file using `torch.save`. To restore the model, `torch.load` is used and the components are reinitialized via `load_state_dict`.

3) POD-NN

The POD-NN model was constructed by combining POD-based reduction with a fully connected neural network that learns the mapping from parameters μ to reduced coefficients. The hyperparameter setup for the POD part is identical to that described earlier, except for the mesh maximum area, which was set to 0.001. The neural network has 3 hidden layers with 30 neurons each in the case of g_1 , and 5 hidden layers with 50 neurons each in the case of g_2 . The activation function is still $\tanh$, the input is the parameter vector (μ_0, μ_1) , while the output is a vector in $\mathbb{R}^N$ representing the coordinates of the solution in the reduced subspace. The PyTorch seed was set to 31 for reproducibility.

The network was trained in a supervised learning setting, using as target the reduced coefficients obtained from the Galerkin projection of the snapshots into the reduced space. The training dataset is the same used in the POD phase and consists of 100 samples. The loss function adopted is the mean squared error between the network's predictions and the true coefficients. Optimization was performed using the Adam algorithm, with an initial learning rate of 10^{-4} in the case of g_1 and 10^{-3} in the case of g_2 , and a `ReduceLROnPlateau` scheduler with factor 0.5 and patience 100. The maximum number of epochs was set to 100,000 for the first case and 500,000 for the second, while the stopping tolerance was kept fixed at 10^{-7} .

For both neural networks, in the PINN and POD-NN models, GPU usage was enabled when available. In our specific case, all training was performed on an NVIDIA GeForce GTX 1650 GPU.

B. Results

In this section the reduced solution seen in § are compared in terms of computational costs and accuracy with respect to the FOM one.

1) Evaluation Metrics

To evaluate the performance of the different approaches tested (PINN, POD, POD-NN), various numerical and computational metrics were used, including absolute error and relative

error. Both are presented below in their continuous and discrete forms.

◇ Absolute Error in L^2 Norm

$$\begin{aligned} & \|u_{\text{model}}(\mu) - u_{\text{FOM}}(\mu)\|_{L^2(\Omega)} \\ & \downarrow \\ & \sqrt{(u_{\text{model}} - u_{\text{FOM}})^T A (u_{\text{model}} - u_{\text{FOM}})} \end{aligned}$$

where u_{FOM} denotes the high-fidelity reference solution, and u_{model} is the approximated solution obtained through one of the reduced models. This metric is used to assess how much the predicted solution deviates on average from the reference solution, i.e., the absolute accuracy over the whole domain.

In the case of the PINN model, the error is not computed solely in the L^2 norm as in the other approaches, but also includes a contribution related to the residual of the PDE. The metric used is therefore a combination of two components: the error on the solution at the degrees of freedom and the error on the PDE residual evaluated in strong form.

$$\|u_{\text{PINN}}(\mu) - u_{\text{FOM}}(\mu)\|_{L^2(\Omega)}^2 + \|\mathcal{L}[u_{\text{PINN}}(\mu)] - f(\cdot, \mu)\|_{L^2(\Omega)}^2$$

In discrete form, the total error is computed as:

$$\sqrt{(u_{\text{FOM}} - u_{\text{PINN}})^T A (u_{\text{FOM}} - u_{\text{PINN}}) + (A_{\text{strong}} R)^T A (A_{\text{strong}} R)}$$

where R is the residual vector of the PDE evaluated at interior points, and A_{strong} is an operator (matrix) that projects the residual onto the degrees of freedom. This type of error reflects not only the difference between the predicted and reference solution, but also how well the neural network satisfies the physics imposed by the differential equation. It is therefore a more coherent metric for evaluating PINN-based models.

◇ Relative Error

$$\begin{aligned} & \frac{\|u_{\text{model}}(\mu) - u_{\text{FOM}}(\mu)\|_{L^2}}{\|u_{\text{FOM}}(\mu)\|_{L^2}} \\ & \downarrow \\ & \frac{\sqrt{(u_{\text{model}} - u_{\text{FOM}})^T A (u_{\text{model}} - u_{\text{FOM}})}}{\sqrt{u_{\text{FOM}}^T A u_{\text{FOM}}}} \end{aligned}$$

The relative error is also used to evaluate the accuracy of the solution. This metric allows comparing the performance of different methods independently of the solution's scale.

◇ Speed-up

$$\text{Speed-up} = \frac{\text{FOM time}}{\text{alternative model time}}$$

where $\langle \text{mod} \rangle$ time refers to the time required to obtain a solution for a new parameter μ using the specified method mod , excluding the training or preprocessing phase. This quantity measures the gain in terms of computational time compared to the full-order finite element solution (FOM). Higher speed-up values indicate greater computational efficiency, making these approaches especially suitable for scenarios requiring rapid or real-time simulations.

All metrics were computed on a test set consisting of $T = 100$ new parameters $\mu = (\mu_0, \mu_1)$, uniformly sampled in the same domain used for training. For each parameter, the high-fidelity reference solution and the corresponding reduced model solution were computed.

The computational time for each method was measured using the `time` library, from which the speed-up was derived. Absolute and relative errors were computed using the inner product induced by the matrix A . Finally, the average values of all three metrics were reported as the overall results of the experiment.

In addition to the computation of quantitative metrics, the accuracy of the reduced-order solutions was also assessed qualitatively through graphical comparisons. Specifically, the solution surfaces obtained using selected reduced models were visually compared with the high-fidelity FOM solution, for fixed values of the parameters μ_0 and μ_1 . This visual inspection helped confirm the consistency of the reduced models in capturing the qualitative behavior of the solution.

The selected parameter values for each case are as follows:

- ◊ For the PINN and POD models in the case with g_1 , and for the POD model in the case with g_2 : $(\mu_0, \mu_1) = (0.45, 0.45)$.
- ◊ For the PINN model in the case with g_2 : $(\mu_0, \mu_1) = (0.75, 0.10)$.

2) Source term independent of parameters

- ◊ confronti tra gli esperimenti
- ◊ Esperimenti nello specifico: confronti risultati con grafici

	POD	PINN	POD-NN
Average absolute error ($\times 10^{-2}$)	0.404	3.895	0.584
Average relative error ($\times 10^{-3}$)	0.877	8.443	1.267
Average speed-up	1.225	70.327	296.372

Table I

RESULTS IN THE CASE WITH SOURCE TERM INDEPENDENT OF PARAMETERS (g_1).

	POD	PINN	POD-NN
Average absolute error ($\times 10^{-2}$)	0.974	16.131	47.972
Average relative error ($\times 10^{-3}$)	1.618	26.064	80.895
Average speed-up	1.176	129.073	668.464

Table II

RESULTS IN THE CASE WITH SOURCE TERM DEPENDENT OF PARAMETERS (g_2).

3) Source term dependent on parameters

4) Comparison of results

IV. CONCLUSIONS

This work shows the behaviour of three different methods for solving a nonlinear parametrized elliptic PDE with homogeneous boundary conditions, taking the full-order model (FOM) as the high-fidelity reference solution. Starting from a classical model order reduction technique, with the expectation that passing through a lower dimensional space would reduce the total computational cost compared to the FOM, we introduced the POD method. Then, we focused on a machine learning approach that exploits both the PDE residual and the boundary conditions, using them directly in the loss function, with the PINN approach. Finally, we presented an hybrid strategy, POD-NN, that combined POD, to extract a reduce basis, with a neural network to approximate the reduced solution.

We tested our models on two cases, exhibiting different characteristics in terms of parameter dependency. In particular, we considered a parameter-independent source term and a parameter-dependent source term. In both scenarios, numerical studies confirmed the expected strengths and limitations of the methods. As we can imagine, all approaches performed better in the parameter-independent source term case than in the parameter-dependent one, when compared the results with the FOM reference. In general, POD provided the most accurate approximations, although the non-affine nature of the PDE kept its computational cost high. PINN achieved substantial speed-ups, but at the cost of a noticeable reduction in accuracy. POD-NN showed mixed behavior. In the parameter-independent case, it clearly offered the best trade-off, combining a very high speed-up with accuracy close to that of POD. In contrast, in the parameter-dependent case, although it still achieved an excellent speed-up, the accuracy did not improve and was overall modest, without providing a particularly strong advantage in terms of precision.

In conclusion, it turned out that POD remains the preferred method when accuracy is the primary goal, while PINN and POD-NN offer remarkable advantages in computational speed. In particular, PINN is suitable when flexibility and mesh-free training are needed, while POD-NN is especially effective in the parameter-independent case, where it achieves both efficiency and reliability.

REFERENCES

- [1] V. Giordano, *Geometria e algebra lineare*. Mondadori Università, 2017.
- [2] S. Berrone and C. Canuto, *Notes of the lecture course numerical methods for partial differential equations*. Non-published notes of the course on "Metodi numerici per le equazioni alle derivate parziali" at the Politecnico di Torino, 2024.
- [3] F. Pichi, B. Moya, and J. S. Hesthaven, "A graph convolutional autoencoder approach to model order reduction for parametrized pdes," *Journal of Computational Physics*, vol. 501, p. 112762, 2024.